

Deterministic Gates for LLM Agents: A Structural Approach to Pass³ Consistency on CAR-bench

Nick Wagner

Independent Researcher

wagnernick1997@gmail.com

Abstract

CAR-bench evaluates agents with Pass³: a task counts only if all three independent trials succeed. This strict metric penalizes stochastic variance more than it rewards peak capability.

We present Glassbox, a deterministic 7-stage runtime shell in which the LLM is responsible only for text generation. All capability checks, policy enforcement, and response-claim verification run as deterministic predicates over a provenance ledger. This is the second iteration of a research line on structural verification: a predecessor, *ast-guard* [Wagner, 2026], validated the guard principle as a live penalty in RL training for code generation. We transfer the same principle to multi-turn agent runtime.

The unmodified Sonnet-4-6 baseline achieves 51.7% (31/60) on a 60-task internal subset. Pass² equals Pass³ for Base and Hallucination, showing that baseline failures are structural rather than stochastic: the same input fails across all three trials. Under an identical deterministic shell, Opus 4.6 achieves Pass³ of 8/18 versus Sonnet 4.6’s 9/18; the architecture, not model scale, determines the outcome. The policy enforcement layer records zero violations (`policy_aut_errors` = 0) across 894 runs. Guards overfit to 60 training tasks (Disambiguation 9.1% on 69 unseen), which we identify, document, and address through a severity-based re-draft mechanism validated via targeted re-verification. Official scores on the hidden test set are pending release on July 29.

1 Introduction

CAR-bench [Kirmayr *et al.*, 2026] evaluates task-oriented vehicle voice agents on three splits: base capability, hallucination resistance, and disambiguation. Its primary metric, Pass³, counts a task only when the agent solves it correctly in all three independent trials. This AND-gate over outcomes means that variance is fatal: an agent that fails inconsistently scores near zero on Pass³ even when its latent ability is high. The public Opus 4.6 baseline achieves Overall = 0.58, Hallu-

cination = 0.48, Disambiguation = 0.46 on the hidden leaderboard test set, confirming that even a strong model leaves headroom in the harder splits.

Multi-turn LLM agents compound this problem. Reliability degrades in underspecified dialogs [Laban *et al.*, 2025], and behavioral reproducibility in multi-step tool-calling pipelines varies widely across configurations [Yagubyan, 2026]. Our own baseline measurements (Section 4.3) confirm the pattern: Pass² equals Pass³ for both Base and Hallucination splits, meaning failures repeat consistently across all three trials. This is not stochastic noise. It is a structural problem.

This work applies one principle throughout: verify the deed, not the narrative. Rather than prompting the LLM to self-correct, each decision point in the agent pipeline is a deterministic predicate that tests a condition over the current conversation state. This is the second iteration of a research line that began with *ast-guard* [Wagner, 2026], a structural gate used as a live penalty during RL training for LLM code generation. In that work, AST-level analysis eliminated reward-hacking at near-zero false-positive rate, because the guard tests a structural representation rather than the model’s self-report. Here we apply the same logic to tool-call preconditions and claim provenance in a multi-turn dialog agent, with the provenance ledger playing the role of the AST.

Our contributions are: (1) a 7-stage deterministic agent shell with a provenance ledger and a policy compiler that enforces 11 of 19 CAR-bench policies fully deterministically; (2) a model-scale ablation showing that the stronger model provides no advantage under an identical shell; (3) zero policy violations across 894 runs, demonstrating null-FP discipline at scale; and (4) transparent documentation of a guard-overfitting failure, its diagnosis, and a validated fix.

2 Related Work

CAR-bench [Kirmayr *et al.*, 2026] builds on τ -bench [Yao and others, 2024], which established realistic user simulators and multi-turn tool interaction as the evaluation setting and introduced Pass^k as a consistency metric.

Two design decisions draw on prior work. First, the SOFT/HARD guard severity distinction follows the Assert/Suggest pattern from DSPy Assertions [Singhvi *et al.*, 2024]: a soft assertion returns feedback and triggers regeneration, a hard assertion halts the pipeline. We adopt this

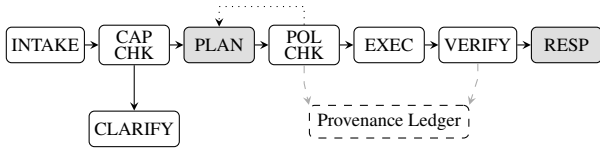


Figure 1: Glassbox 7-stage pipeline. **Shaded stages** (PLAN, RESP) invoke the LLM at temperature = 0; all other stages are deterministic code. Guard stages (POL CHK, VERIFY) read from and write to the provenance ledger (dashed). The dotted arc is the bounded re-plan loop (at most two rounds). CLARIFY terminates the current turn; the next user message restarts at INTAKE.

semantics directly, using SOFT for re-draft (at most twice) and HARD for deterministic replacement.

Second, the Auditor is a deterministic code module rather than a second LLM call. MARCH [Li *et al.*, 2026] shows that LLM-as-judge suffers from confirmation bias: the verifier reproduces the generator’s errors because it shares the same parametric knowledge. Deliberate information asymmetry breaks this cycle. Our design takes the asymmetry to its limit: the verifier has no generative capacity at all, so it cannot reproduce an LLM fabrication.

The motivation for targeting structural failures follows from two observations. Multi-turn reliability degrades in underspecified settings [Laban *et al.*, 2025]. Separately, our baseline measurement shows that $\text{Pass}^2 = \text{Pass}^3$ on Base and Hallucination: the unmodified model either solves a task consistently or fails it consistently. Guards on structural predicates address exactly this failure mode.

The methodological predecessor is *ast-guard* [Wagner, 2026], which applied predicate checks on AST representations as a live training penalty. That work first established that structural guards can achieve near-zero false-positive rate in practice. The current work transfers the same design principle to agent runtime.

3 Architecture

3.1 Ledger and State Machine

The agent maintains a provenance ledger that records every piece of information in the conversation: tool call arguments and results, user utterances, and policy defaults. Every factual claim in the final response must be traceable to a ledger entry.

Control flow follows a 7-stage state machine (Figure 1): INTAKE, CAPABILITY_CHECK, CLARIFY or PLAN, POLICY_CHECK, EXECUTE, VERIFY, and RESPOND. The PLAN loop is bounded to at most two rounds and uses idempotent call IDs to prevent duplicate tool execution. LLM calls occur only in PLAN (planning) and RESPOND (drafting); temperature is set to 0.

3.2 Policy Compiler

CAR-bench publishes 19 policies that govern agent behavior. We classify each by testability at runtime: **Class A** (fully deterministic, 11 policies), **Class B** (deterministic precondition with a semantic residual, 5 policies), and **Class C** (inherently semantic, 3 policies).

Class A policies are compiled into a declarative rule table (policies.py: RULES) with five rule types:

state_precondition (block if a required state is not met), state_companion (inject the missing prerequisite tool call), value_bound (reject out-of-range arguments), requires_confirmation (block without explicit user “yes” in the ledger), and no_parallel (prevent concurrent state-changing calls). No tool name appears in the control-flow code; names exist only in the data. This design makes Class A violations structurally impossible.

The guard principle throughout is: $\text{precondition}(\text{state}) \rightarrow \text{check}$. Before any state-changing action, the relevant predicate is tested against the ledger. Failure triggers injection of missing observations, a block, or a re-draft request, depending on the policy class and severity level.

3.3 Fabrication Guard and Auditor

The Fabrication Guard runs at two checkpoints. Before execution (C2), it tests that every argument in a proposed tool call has a traceable source in the ledger. Before the response (C5), it uses a structured LLM extraction to surface latent factual claims in the draft and checks each against the ledger corpus.

The Auditor folds a forced self-check into the existing VERIFY draft call. The LLM must enumerate every factual claim with its verbatim ledger source before writing the response text. The Auditor parses this enumeration deterministically: only numerical values are falsifiable at code level. Unsupported numerical claims are replaced with honest admissions of inability; string paraphrases pass unchanged. This zero-FP discipline is the consequence of Lesson 1a: the LLM provides candidate claims, code decides which are supported.

3.4 Disambiguation Engine

The Disambiguation Engine runs as a pre-flight guard inside the PLAN loop, not as a separate pre-plan step. This placement is necessary because preference lookups (priority 2 in the resolution cascade) can only be evaluated after the tool call completes. For each ambiguous slot, the engine applies a 5-level cascade: policy default, explicit user value, learned preference (via `get_user_preferences`), heuristic default, and user query. At priority 2, the engine injects a preference lookup, defers the state-changing action, and applies the result in the next planning round.

3.5 Guard Severity (Phase 1)

Phase 1 introduced SOFT and HARD severity levels following the Assert/Suggest distinction from DSPy [Singhvi *et al.*, 2024]. A SOFT guard sends structured feedback to the LLM and triggers a re-draft, at most twice. A HARD guard blocks the action and replaces the offending output deterministically. This distinction fixed a documented regression in which the initial guards over-blocked valid base tasks (Section 4.3). RC-Tool-Injection, also part of Phase 1, adds REQUIRES_CONFIRMATION confirmation steps additively to the plan, without modifying the existing planning path.

What is active in the submitted agent (commit 60310a1). The core architecture (Fabrication Guard, Policy Compiler, Ledger, Disambiguation Engine, Auditor), Phase 1 (Guard

Severity, RC-Tool-Injection), and Phase 2 (few-shot anchors in planning; Multi-Stop-Message-Enforcer). Phase 2 is included but the Enforcer component did not pass its targeted re-verification (see Section 5). Phase 3 (Response Coherence Layer) was not included.

4 Experiments

Setup. Agent model: `claude-sonnet-4-6`, temperature = 0. User simulator and judge: `gemini-2.5-flash`. Evaluation subset (Subset-L): 60 tasks from the training split (23 Base, 20 Hallucination, 17 Disambiguation), 3 independent trials each. Primary metric: Pass³ (task-weighted).

4.1 Model-Scale Ablation

To test whether model scale determines outcomes under a fixed shell, we ran the same Glassbox configuration with Claude Sonnet 4.6 and Claude Opus 4.6 on an 18-task subset (4 Base, 4 Hallucination, 10 Disambiguation), 3 trials each (runs 20260711-103751 and 20260711-182802, both at commit 66b116f).

Sonnet achieves Pass³ of 9/18 (50.0%); Opus achieves 8/18 (44.4%). Five tasks are solved 3/3 by both models: `base_10`, `base_32`, `dis_0`, `hall_10`, `hall_16`. One task (`dis_22`) fails 0/3 under both.

A one-task difference on 18 tasks is not statistically significant. The relevant finding is the direction: the stronger model provides no advantage under an identical deterministic shell. Architecture, not model scale, determines the outcome. This directly addresses the innovation criterion of performance relative to model investment.

4.2 Policy Enforcement Guarantee

The `policy_aut_errors` counter records evaluator-detected policy violations. Across 25 archived experiment runs (894 total trials, spanning 3-trial smoke tests to 207-trial full-split runs), this counter is zero. Verification was performed by script against all archived JSON artifacts.

Zero violations at this scale confirms that the Class A enforcement design, which makes violations structurally impossible rather than probabilistically unlikely, achieves null-FP discipline in practice.

4.3 Subset-L Ablation

Table 1 shows three configurations on Subset-L.

Baseline (naked Sonnet, run 20260721-061226). The unmodified `claude-sonnet-4-6` (AGENT.CLASS=baseline) achieves 51.7% (31/60) Pass³ at a cost of \$0.158 per trial. Examining consistency: Pass² equals Pass³ for both Base (0.609) and Hallucination (0.400). This means that on every task the baseline fails, it fails all three trials. There are no 2/3 near-misses: errors are structural, not stochastic. On `dis_55`, the most complex disambiguation task, the baseline requires 86 LLM calls and \$3.10 across three trials.

This failure profile is the precise target of the deterministic shell: if errors repeat mechanically across trials, a guard that catches the structural cause can flip the entire task from 0 to 3/3.

Configuration	Overall	Base	Hall.	Dis.
Baseline (naked)	51.7%	60.9%	40.0%	52.9%
Glassbox pre-fix	45.0%	56.5%	35.0%	41.2%
Submitted (Ph. 0–2)	<i>pending</i>			
Baseline = Sonnet-4-6, AGENT_CLASS=baseline.				
Glassbox pre-fix = initial guards before severity fix.				

Table 1: Subset-L ablation (Pass³, task-weighted, train split). Cost/trial: Baseline \$0.158, Glassbox \$0.219. Not comparable to hidden-test leaderboard. Official scores pending July 29.

Glassbox, pre-fix (run 20260714-051201). After implementing the initial guard package (Fix-Paket L, commit 883db7b), overall Pass³ dropped to 45.0% (27/60) at \$0.219 per trial. Base fell from 60.9% to 56.5%; Hallucination from 40.0% to 35.0%. Root-cause diagnosis: these pre-Phase-1 guards over-blocked valid base tasks where predicate thresholds were calibrated too strictly on training-distribution inputs. This is a documented guard-overfitting regression. We record it here as a finding, not as a failure of the approach: the regression is recoverable because it has a structural cause.

Phase-1 fix and submitted agent (commit 60310a1). Phase 1 introduced SOFT/HARD severity levels to address the over-blocking. Targeted re-verification confirmed the fix: `base_2` recovered from 0/3 to 3/3 Pass³; all five sentinel tasks remained stable at Pass³. A full Subset-L run after the Phase-1 fix was not conducted due to compute budget. The submitted agent, which includes Phase 1 and Phase 2, was evaluated on the official hidden test set.

Official hidden-set scores are pending release on July 29; Table 1 will be updated with those results.

Cost per solved task. Raw cost per trial favors the baseline (\$0.158 vs. \$0.219). Normalized by Pass³ rate, the comparison shifts: at 51.7% the baseline spends \$0.306 per successfully solved task; at 45.0% the pre-fix Glassbox spends \$0.487. The gap reflects the guard-overfitting regression documented above, not an inherent property of the architecture. Without guards, the baseline enters long recovery loops on hard tasks: `dis_55` requires 86 LLM calls and \$3.10 across three trials. The deterministic shell bounds this worst case. Ledger-based caching of repeated tool calls with known provenance is a direct path to reducing per-trial cost in future iterations.

4.4 Generalization

To test generalization, we evaluated the pre-fix Glassbox on 69 tasks not seen during guard development (run 20260713-040017). Overall Pass³ was 44.9% (31/69): Base 50.0%, Hallucination 53.6%, Disambiguation 9.1%.

The Disambiguation collapse (9.1% on 69 unseen vs. 41.2% on 60 seen) indicates that disambiguation predicates were tuned on training-split patterns. This finding prompted a sealed holdout protocol for all subsequent development: guard changes are calibrated only on the training subset, and the holdout set is touched at most once per major phase.

5 Limitations and Future Work

Phase 2 Enforcer. The Multi-Stop-Message-Enforcer (Phase 2) did not pass its targeted re-verification (0/3 on enforcer-specific tasks; one sentinel task regressed). It is included in the submitted agent as “implemented, verification pending.” A full Phase 2 Mini-Verify cycle is the first outstanding task.

Phase 3: Response Coherence Layer. Phase 3 (RCL) was not included in the submitted agent. RCL checks that every state-change claim in the response is backed by a SUCCESS tool result; unsupported claims are replaced deterministically. The code is written and unit-tested; it was not submitted because no Mini-Verify run confirmed zero regression on sentinel tasks.

Parser generalizability. The policy compiler is tied to the published CAR-bench policy schema. Porting it to a different benchmark requires rewriting the Class A rule table and re-running the null-FP verification cycle. The architecture supports this, but the effort is non-trivial.

Class C policies. Three of 19 policies remain inherently semantic and are addressed only through VERIFY-prompt guidance. Full deterministic coverage would require structured output extraction for these cases.

Metric vs. experience tradeoff. HARD-block guards maximize Pass³ by refusing policy-violating requests outright rather than issuing a best-effort response. In a production setting, this tradeoff requires explicit calibration.

Resource scope. This work was conducted by a single independent researcher without institutional compute access. Full post-fix Subset-L evaluation runs and broader hyperparameter search were limited by budget.

6 Conclusion

Under an identical deterministic shell, Claude Opus 4.6 and Sonnet 4.6 achieve near-identical Pass³ results (8/18 vs. 9/18). Architecture is the primary lever, not model scale. Zero policy violations across 894 runs confirm that structural enforcement achieves null-FP discipline at scale. The documented guard-overfitting failure and its targeted fix illustrate why transparent measurement protocols, including sealed holdout sets and explicit regression tracking, matter more than benchmark-optimized reporting.

Acknowledgements

The author used Claude (Anthropic) as a research, analysis, and writing assistant throughout this project, including in the preparation of this report. All experiments, architecture decisions, and conclusions are the author’s own. This work was conducted by a single independent researcher without institutional affiliation or dedicated compute budget.

References

[Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world

uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.

[Laban *et al.*, 2025] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation, 2025.

[Li *et al.*, 2026] Zhuo Li, Yupeng Zhang, Pengyu Cheng, Jiajun Song, Mengyu Zhou, Hao Li, Shujie Hu, Yu Qin, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. MARCH: Multi-agent reinforced self-check for LLM hallucination, 2026.

[Singhvi *et al.*, 2024] Arnav Singhvi, Manish Shetty, Shangyin Tan, Christopher Potts, Koushik Sen, Matei Zaharia, and Omar Khattab. DSPy assertions: Computational constraints for self-refining language model pipelines, 2024.

[Wagner, 2026] Nick Wagner. ast-guard: Deterministic AST-level gate against reward hacking in RL training. <https://github.com/Nick-is-building/ast-guard>, 2026. Software.

[Yagubyan, 2026] Abel Yagubyan. How consistent are LLM agents? Measuring behavioral reproducibility in multi-step tool-calling pipelines, 2026.

[Yao and others, 2024] Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.